

Introduction to R & Environmental Data

SHM 377 Hazardous Materials Management

Yoni Rodriguez

Table of contents

Virtual Lab 01	3
Introduction to R & Environmental Data	3
Step 1: Install R and RStudio	3
What Is R?	4
What Is RStudio?	4
Windows	5
macOS	5
Check Your Installation	6
Step 2: Download the Dataset from GitHub	6
Step 3: Create Your SHM 377 Folders	7
File Naming	8
Step 4: Open RStudio and Create an R Script	8
Create an R Script	8
Option 1: Use the Menu	8
Option 2: Use the New File Button	9
Save the Script Immediately	9
Step 5: Learn the Four RStudio Panes	9
Source Pane	9
Console Pane	10
Environment / History Pane	10
Files / Plots / Packages / Help / Viewer Pane	10
Use the Files Pane	10
Step 6: The Console and R Script Have Different Jobs	11
Try a Command in the Console	11
Type the Same Command in Your R Script	12
Run Code from the Script	12
Step 7: Start Your R Script	13
What Does # Mean?	13

Step 8: Make Sure R Can Find Your Files	13
Check the Working Directory	14
See What Files R Can Find	15
If R Is Looking in the Wrong Folder	15
Step 9: Import the Dataset	17
Read the Command	17
Command	17
Function	17
Argument	18
Object	18
Assignment Operator	18
Step 10: Inspect the Dataset in the Console	18
Look at the First Rows	18
Look at the Last Rows	19
See the Variable Names	19
Check the Dimensions	19
Examine the Structure	19
Step 11: Understand the Monitoring Dataset	20
Step 12: Prepare the Date and Time Variable	20
Step 13: Check the Measurements	21
Unusual Values	21
Step 14: Describe the Data	22
Step 15: Visualize Concentration Through Time	22
Reading the plot() Function	23
Step 16: Look Before Explaining	23
Step 17: Interpret What the Data Show	24
What Can We Examine?	24
What Can We Not Conclude?	24
Step 18: Document Your Interpretation in the R Script	24
Step 19: Export and Save Your Work	25
Export the Plot	25
Review Your R Script	26
Why Units Matter	27
The SHM 377 Data Workflow	28
Connecting Data to Hazardous Materials Management	28

You Do Not Need to Memorize Everything	29
What You Accomplished	30
The Main Idea	30
Where We Go Next	31

Virtual Lab 01

Introduction to R & Environmental Data

This first virtual lab begins at the beginning.

We will install the software, organize our files, create an R script, learn the RStudio workspace, import a real environmental monitoring dataset, inspect the data, create a visualization, and interpret what we see.

! Virtual Lab 01 Submission and Credit

Virtual Lab 01 is graded for **completion**.
To receive full credit:

1. **Show me that your code runs on your local computer during class.**
2. **Complete the interpretation section in your R script.**
3. **Save your R script as `virtual-lab-01.R`.**
4. **Upload `virtual-lab-01.R` directly to Canvas.**

If you complete these requirements, you will receive **full credit for Virtual Lab 01**.
The purpose of this first lab is to make sure you can successfully complete the basic workflow we will use throughout the quarter.

! Complete the Steps in Order

Do not skip ahead. Complete each numbered step in order. Later steps depend on files and objects created in earlier steps.

The goal is not to become a programmer today.

The goal is to learn a repeatable workflow for using R to investigate and communicate environmental and hazardous materials data.

Step 1: Install R and RStudio

We will use two pieces of software in this course: **R** and **RStudio**.

They work together, but they are not the same thing.

R is the program that performs the analysis. **RStudio** is the workspace we use to interact with **R**.

```
R
Performs calculations
Runs commands
Analyzes data

    ↓ works through

RStudio
Provides the workspace
Write and save scripts
Run commands
View files
View data
Create graphs
```

What Is R?



R is a programming language and analytical environment used for data analysis, statistics, visualization, and reproducible research.

When we ask **R** to calculate a mean, summarize a dataset, or create a graph, **R is the software actually performing those operations.**

[R Project](#)

What Is RStudio?



RStudio is an integrated development environment, or IDE, that provides an organized workspace for working with **R**.

RStudio allows us to write and save code, run commands, view data, navigate files, create graphs, and access help.

RStudio Desktop from Posit

! Installation order matters

Install the software in this order:

1. **Install R**
2. **Install RStudio Desktop**
3. **Open RStudio**
4. **Confirm that RStudio can find and run R**

RStudio provides the interface, but R performs the computations.

Windows

1. Go to the [R Project](#).
2. Select **CRAN**.
3. Scroll down to the **USA** section.
4. Select the **Oregon State University** CRAN mirror.

USA

<http://ftp.usgs.iu.edu/CRAN/>
<https://mirror.its.umich.edu/cran/>
<https://cran.wustl.edu/>
<https://archive.linux.duke.edu/cran/>
<https://cran.case.edu/>
<https://ftp.osuosl.org/pub/cran/>
<https://lib.stat.cmu.edu/R/CRAN/>
<https://cran.mirrors.hoobly.com/>
<https://mirrors.nics.utk.edu/cran/>
<https://mirror.chpc.utah.edu/pub/cran/>

Indiana University
University of Michigan, Ann Arbor, MI
Washington University, St. Louis, MO
Duke University, Durham, NC
Case Western Reserve University, Cleveland, OH
Oregon State University
Statlib, Carnegie Mellon University, Pittsburgh, PA
Hoobly Classifieds, Pittsburgh, PA
National Institute for Computational Sciences, Oak Ridge, TN
University of Utah

5. Select **Download R for Windows**.
6. Download and install the current version of R.
7. Go to the [RStudio Desktop download page](#).
8. Download and install the free version of RStudio Desktop.
9. Open **RStudio**.

💡 Which CRAN Mirror Should I Choose?

For this course, select the **Oregon State University** mirror shown above.

A CRAN mirror is simply a server that provides the R installation files. There are many mirrors because the same files are hosted at different locations.

You do not need to compare the different mirrors. For our class, use **Oregon State University**.

macOS

1. Go to the [R Project](#).
2. Select **CRAN**.
3. Choose a CRAN mirror.
4. Select **Download R for macOS**.

5. Read the installer descriptions and choose the version appropriate for your Mac.
6. Install R.
7. Go to the [RStudio Desktop download page](#).
8. Download and install RStudio Desktop.
9. Open **RStudio**.

macOS users

Do not guess which installer you need. If you are unsure which version is appropriate for your Mac, ask for help before continuing.

Check Your Installation

When RStudio opens, locate the **Console**.

At the `>` prompt, enter:

```
2 + 2
```

R should return:

```
[1] 4
```

If you see `[1] 4`, R and RStudio are communicating correctly.

Step 2: Download the Dataset from GitHub

We will use GitHub throughout SHM 377 to access lecture materials, datasets, and other course resources.

For this lab, we need:

`winter-smoke-2019-12-27_to_2019-12-31.csv`

A **CSV** is a data file. CSV stands for **comma separated values**. For this lab, the CSV contains the environmental measurements we will analyze in R.

Use the SHM 377 GitHub Guide

Before continuing, open the **How to Download SHM 377 Course Materials from GitHub** guide in the **Week 1 module on Canvas**.

The guide walks you through how to:

1. Find the SHM 377 GitHub repository.
2. Navigate to the correct week and lecture folder.
3. Download course materials.
4. Download the CSV dataset.
5. Move the CSV into your SHM 377 course folder.

6. Find the dataset using the RStudio Files pane.

You do not need to learn GitHub programming or Git commands for this course.

For this lab, follow the guide until you have downloaded:

```
winter-smoke-2019-12-27_to_2019-12-31.csv
```

Your browser will normally place the file in your computer's **Downloads** folder.

Do not open the CSV in Excel yet.

We will organize the file before importing it into R.

Step 3: Create Your SHM 377 Folders

Before we begin the analysis, create a permanent location for your SHM 377 course files on your computer.

A simple organization is:

```
SHM-377/  
  virtual-lab-01/
```

Move the CSV you downloaded from your computer's **Downloads** folder into `virtual-lab-01`.

For example:

```
SHM-377/  
  virtual-lab-01/  
    winter-smoke-2019-12-27_to_2019-12-31.csv
```

Your exact folder structure may look different. That is okay.

The important part is that you know where the CSV is stored and that it is inside the folder you are using for your RStudio work.

! Your Downloads folder is not your project folder

Your browser may place downloaded files in **Downloads** automatically.

Move the CSV into the `virtual-lab-01` folder you created for this course. Keeping related files together makes your work easier to find and reduces file path problems.

File Naming

Use names that tell you what a file contains.

Good:

```
winter-smoke-2019-12-27_to_2019-12-31.csv  
virtual-lab-01.R
```

Avoid:

```
data.csv  
final.R  
final-final.R  
stuff.csv
```

Clear file organization is part of a reproducible professional workflow.

Step 4: Open RStudio and Create an R Script

Open **RStudio**.

When RStudio first opens, your screen may not look exactly like the example used in class.

i You may initially see only three panes

Nothing is wrong.

The fourth pane appears when you open a file in the **Source** pane.

Create an R Script

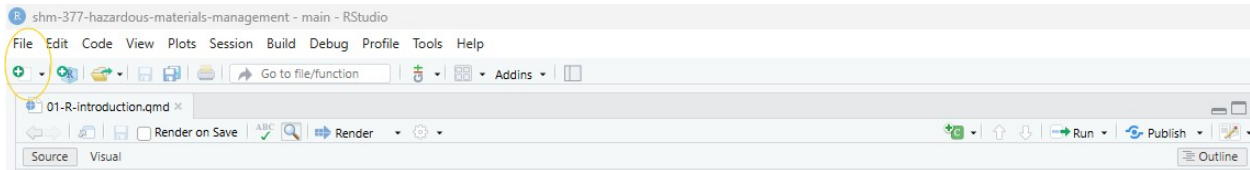
There are two ways to create a new R script.

Option 1: Use the Menu

1. Select **File**.
2. Select **New File**.
3. Select **R Script**.

Option 2: Use the New File Button

You can also select the **New File** button in the upper left corner of RStudio and choose **R Script**.



After you create the R script, a new pane will appear in the upper left portion of RStudio.

This is the **Source pane**.

An R script is a file where you write and save R commands.

Save the Script Immediately

Select **File** → **Save As**.

Navigate to:

```
SHM-377/  
  virtual-lab-01/
```

Save the script as:

```
virtual-lab-01.R
```

Your lab folder should now contain:

```
SHM-377/  
  virtual-lab-01/  
    winter-smoke-2019-12-27_to_2019-12-31.csv  
    virtual-lab-01.R
```

The `.R` file is your R script.

Step 5: Learn the Four RStudio Panes

Now that an R script is open, RStudio should display the four main areas we will use.

Source Pane

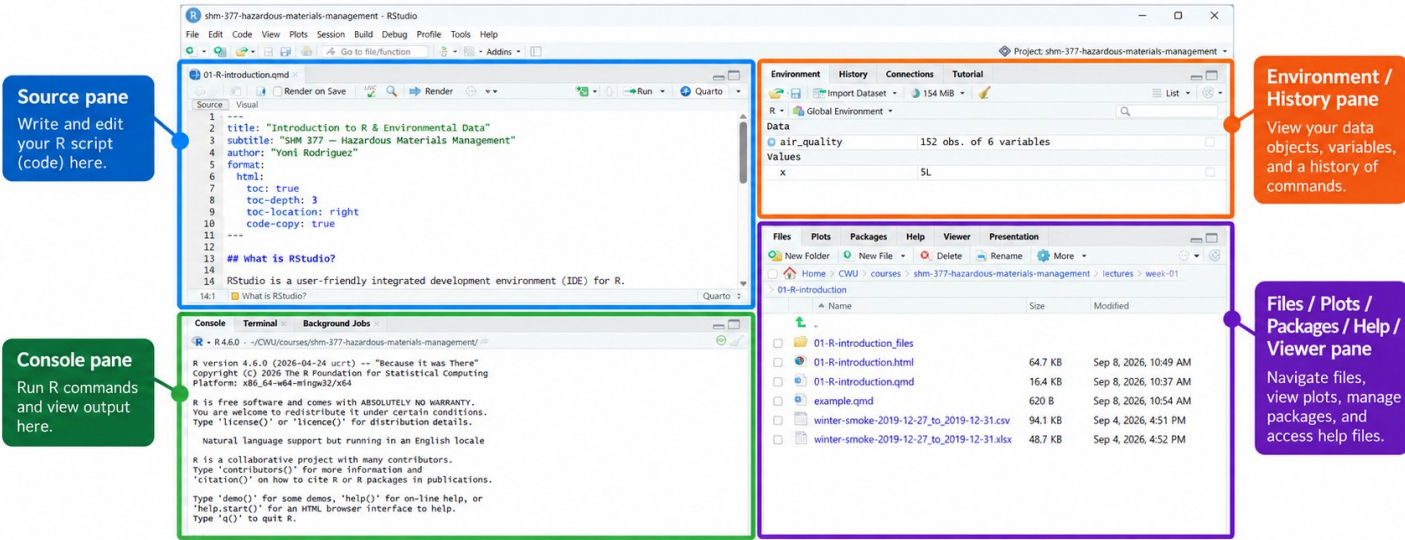
The **Source pane** is where your R script is open.

Use it to write and save code that you want to keep.

2.2 What is RStudio?

RStudio is a user-friendly integrated development environment (IDE) for R. It makes it easier to write code, run analyses, visualize data, and manage your files and packages. The RStudio interface is divided into four main panes:

1. **Source pane (top left):** where you write and edit your R script (code).
2. **Console pane (bottom left):** where R runs your commands and displays output.
3. **Environment/History pane (top right):** shows your data objects, variables, and a history of past commands.
4. **Files/Plots/Packages/Help/Viewer pane (bottom right):** used to navigate files, view plots, manage packages, and view help files.



Console Pane

The **Console pane** is where R executes commands and displays output.

You can also type exploratory commands directly into the Console.

Environment / History Pane

The **Environment / History pane** shows your data objects, variables, and a history of past commands.

Later, after we import the dataset, you should see it appear in the Environment.

Files / Plots / Packages / Help / Viewer Pane

The **Files / Plots / Packages / Help / Viewer pane** allows you to navigate files, view plots, manage packages, and access help files.

Use the Files Pane

Select the **Files** tab and navigate through the folders on your computer.

Find:

```
SHM-377/  
virtual-lab-01/
```

You should see your CSV and R script.

The folder exists on your computer. The Files pane lets you navigate those same folders from inside RStudio.

Step 6: The Console and R Script Have Different Jobs

The Console and R script both allow us to work with R, but they serve different purposes.

Console	R Script
Try something	Save your work
Explore	Build an analysis
Check something	Keep important commands
Immediate	Reusable
Not automatically saved as your analysis	Saved as an .R file

Console	R Script
Try something	Save your work
Explore	Build an analysis
Check something	Keep important commands
Immediate	Reusable
Not automatically saved as your analysis	Saved as an .R file

Try a Command in the Console

Enter:

```
2 + 2
```

R should immediately return:

```
[1] 4
```

Type the Same Command in Your R Script

Now type this in the **Source pane**:

```
2 + 2
```

What happened?

Nothing happened yet.

Writing code and running code are two different actions.

Writing a command in an R script saves the instruction. R does not perform that instruction until you run it.

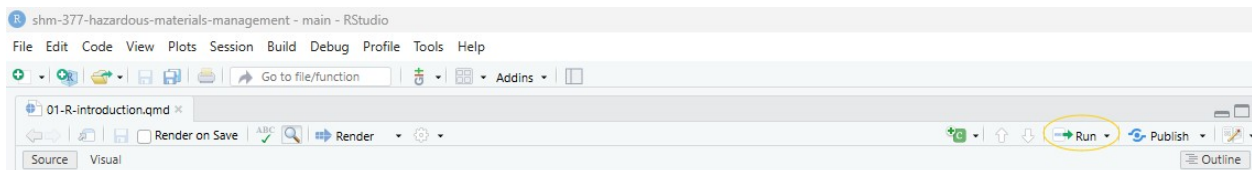
Run Code from the Script

Writing code in the Source pane does not automatically run it.

To run a line from your R script:

1. Click anywhere on the line you want to run.
2. Select the **Run** button at the top of the Source pane.

The **Run** button is highlighted in the image below.



You can also use the keyboard shortcut:

Windows: Ctrl + Enter

macOS: Command + Enter

The command is sent from your R script to the **Console**, where R executes it.

💡 Script → Run → Console

Think of the process this way:

Write the command in your R script

↓

Select Run

↓

Command is sent to the Console

↓

R executes it

Step 7: Start Your R Script

Delete the practice `2 + 2` line from your script.

At the top of `virtual-lab-01.R`, type:

```
# Virtual Lab 01
# Your Name
# Date
```

What Does # Mean?

In an R script, `#` begins a **comment**.

R does not execute the text after `#` on that line.

Comments help people understand and organize code.

Add another comment:

```
# Import Data
```

Your script should now begin something like:

```
# Virtual Lab 01
# Your Name
# Date

# Import Data
```

💡 Write code for your future self

A good R script should not only run correctly. It should also help you understand what you did when you return to the analysis later.

Step 8: Make Sure R Can Find Your Files

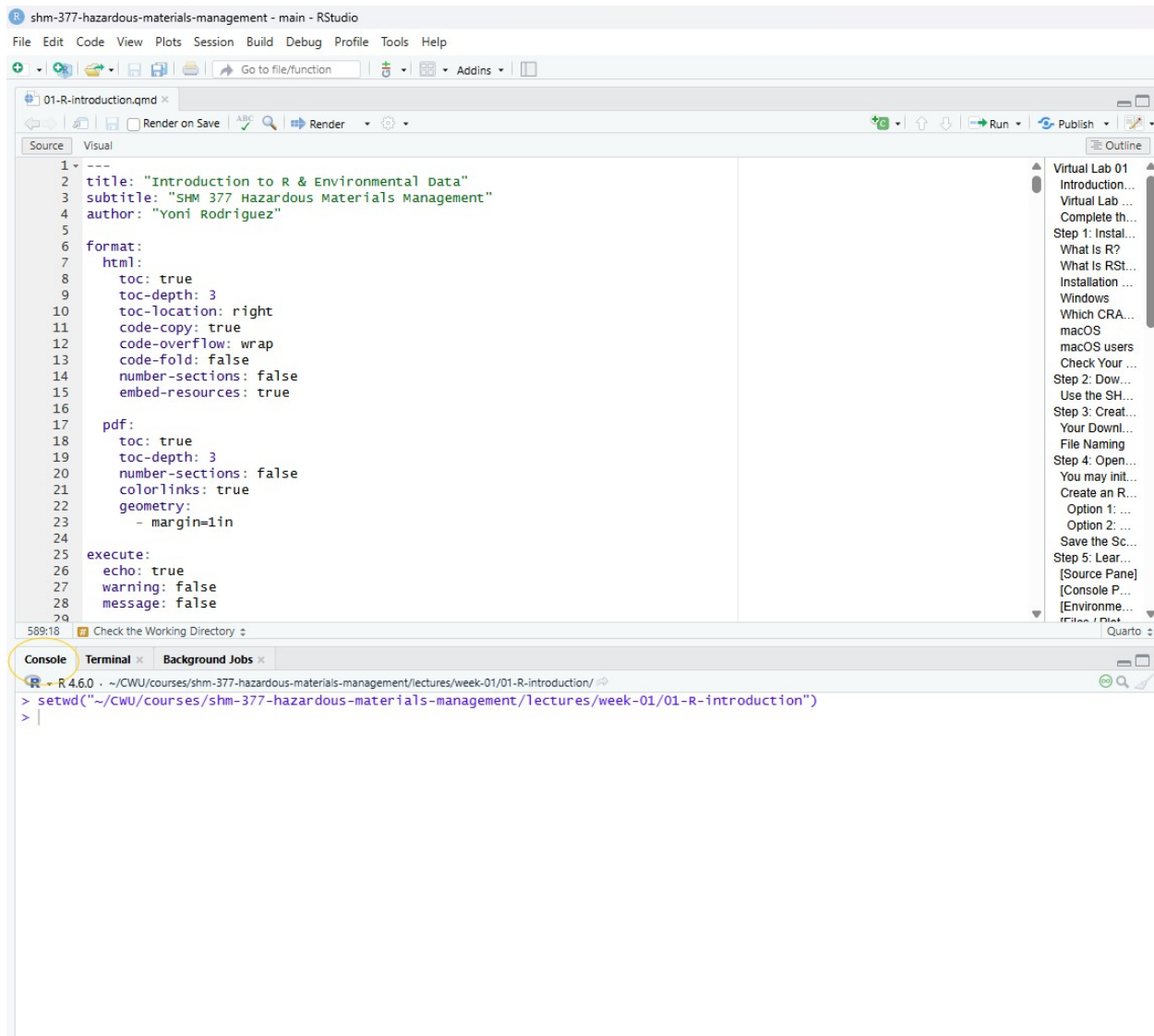
Before importing the CSV, we need to make sure R is looking in the correct folder.

The location R is currently using is called the **working directory**.

Check the Working Directory

We will run the next command directly in the **Console**.

The **Console** pane is the lower left pane in RStudio, highlighted in green below.



Click next to the `>` prompt in the Console and enter:

```
getwd()
```

Function: `getwd()`

Purpose: Show the current working directory.

R will display the folder it is currently using.

Ask:

Is R looking inside my `virtual-lab-01` folder?

See What Files R Can Find

In the Console, run:

```
list.files()
```

Function: `list.files()`

Purpose: Show files in the current working directory.

You want to see:

```
winter-smoke-2019-12-27_to_2019-12-31.csv
```

If the CSV appears in `list.files()`, R can see it.

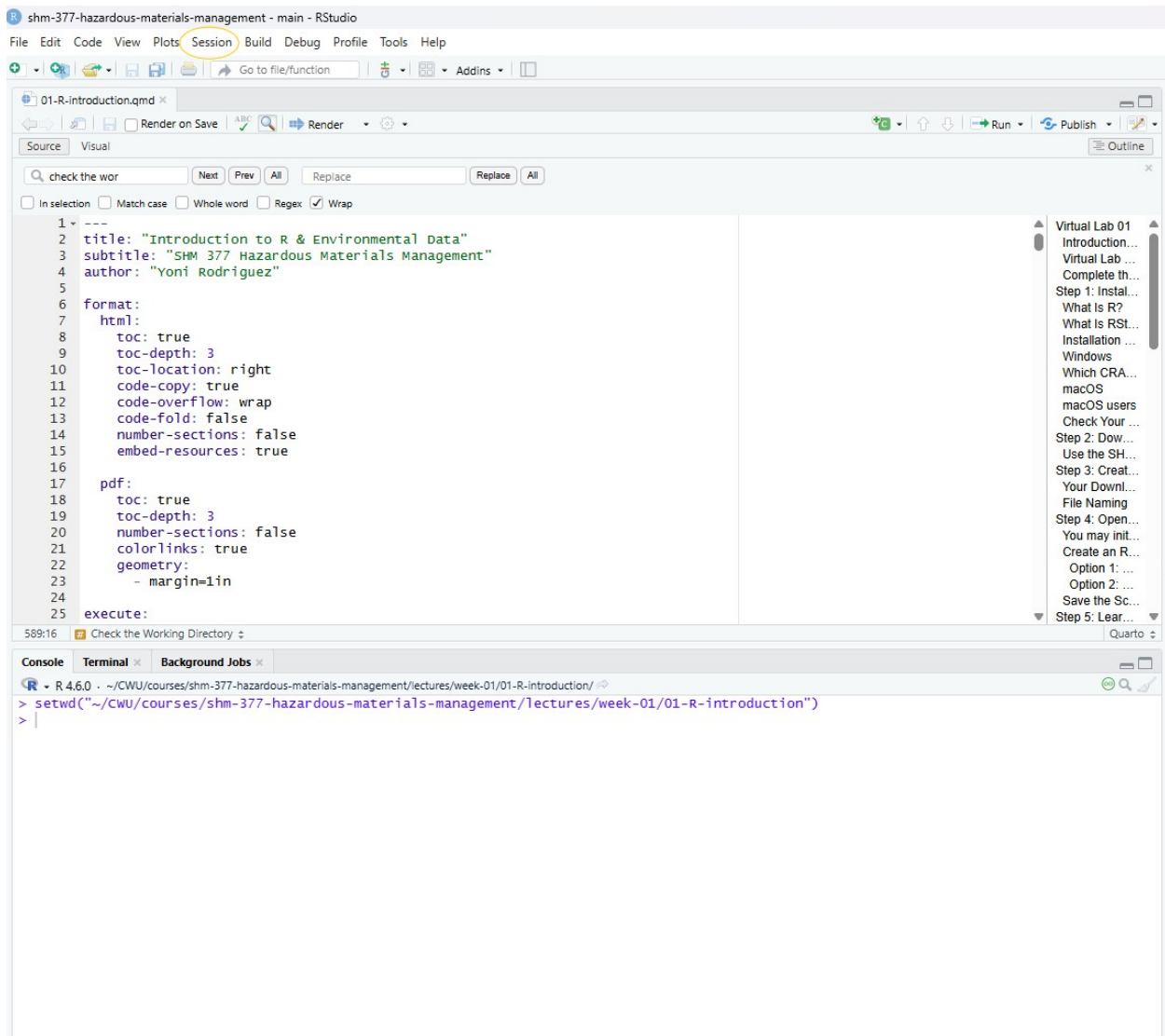
If R Is Looking in the Wrong Folder

Make sure `virtual-lab-01.R` is open in the **Source pane**.

Then:

1. Select **Session** from the RStudio menu.
2. Select **Set Working Directory**.
3. Select **To Source File Location**.

The menu path is shown below:



This tells R to use the folder containing your open R script as the working directory.

Now return to the **Console** and check again:

```
getwd()
list.files()
```

You should now see that R is looking inside your `virtual-lab-01` folder and that your CSV appears when you run `list.files()`.

! When R cannot find a file

Ask two questions:

1. **Where is R looking?**
2. **Is the file actually there?**

`getwd()` and `list.files()` can solve many file import problems.

Step 9: Import the Dataset

Now we are ready to add the first meaningful analysis command to our **R script**.

Under **# Import Data**, enter:

```
winter_smoke <- read.csv(  
  "winter-smoke-2019-12-27_to_2019-12-31.csv"  
)
```

Run the command.

After it runs successfully, `winter_smoke` should appear in the **Environment** pane.

Read the Command

```
winter_smoke <- read.csv("winter-smoke-2019-12-27_to_2019-12-31.csv")  
  ↑           ↑           ↑  
  object      function    argument  
      ↑  
  assignment operator
```

Command

A **command** is an instruction we give R.

The entire line is a command.

Function

A **function** is a tool in R that performs a task.

Here, the function is:

```
read.csv()
```

Its job is to import a CSV file.

Argument

An **argument** gives a function information it needs to perform its task.

Here, the argument is the name of the CSV file:

```
"winter-smoke-2019-12-27_to_2019-12-31.csv"
```

Object

An **object** is something stored in R.

Here, the imported dataset is stored as:

```
winter_smoke
```

Assignment Operator

The symbol:

```
<-
```

is the **assignment operator**.

You can read the command from right to left:

Read this CSV file and store the result in an object called `winter_smoke`.

Step 10: Inspect the Dataset in the Console

We imported the dataset using our **R script** because importing it is an important part of the analysis we want to preserve.

Now we will investigate the dataset in the **Console**.

Look at the First Rows

Run in the Console:

```
head(winter_smoke)
```

Read the command:

```
head(winter_smoke)
↑      ↑
function argument
```

The entire line is a **command**.

`head()` is the **function**.

`winter_smoke` is the **argument** supplied to the function.

Purpose: `head()` shows the first several rows of a dataset.

Look at the Last Rows

```
tail(winter_smoke)
```

Purpose: `tail()` shows the last several rows.

See the Variable Names

```
names(winter_smoke)
```

We expect:

```
date_time  
concentration
```

Check the Dimensions

```
dim(winter_smoke)
```

R reports:

```
rows columns
```

The rows are observations. The columns are variables.

Examine the Structure

```
str(winter_smoke)
```

`str()` shows information such as variable names, variable types, and example values.

This is one of the most useful checks immediately after importing a dataset.

Step 11: Understand the Monitoring Dataset

Now that we have successfully imported and inspected the file, we can focus on the environmental question.

What happened to ambient particle concentrations during five days of winter monitoring?

The dataset contains approximately 3,600 observations collected from **December 27 through December 31, 2019**.

Variable	Meaning
<code>date_time</code>	When the measurement was collected
<code>concentration</code>	Measured particle concentration

The measurements were collected using a **TSI DustTrak**.

Particle concentration was measured in **milligrams per cubic meter (mg/m³)**.

The instrument measured particle concentration. It did not identify the exact source of every particle.

Measurement does not equal source attribution.

Step 12: Prepare the Date and Time Variable

Our inspection may show that `date_time` was imported as text.

Because correcting the variable type is part of our analysis, save this command in the **R script**.

Add a section comment:

```
# Convert Date and Time
```

Then add:

```
winter_smoke$date_time <- as.POSIXct(  
  winter_smoke$date_time  
)
```

Function: `as.POSIXct()`

Purpose: Convert date and time information into a format R recognizes as date and time data.

Run the command.

Then return to the Console and check:

```
str(winter_smoke)
```

Did the variable type change?

```
Inspect in the Console
  ↓
Identify a problem
  ↓
Write the needed change in the R script
  ↓
Run the change
  ↓
Check again in the Console
```

Do not assume your code worked. Check.

Step 13: Check the Measurements

Before calculating averages or making graphs, inspect the measurement range.

Run these exploratory checks in the Console:

```
min(winter_smoke$concentration)
max(winter_smoke$concentration)
```

The observed concentration range is approximately:

```
0.000 to 0.288
```

Ask:

Do these measurements appear plausible?

Unusual Values

An unusual value is not automatically an error.

It could represent a real environmental event, a nearby source, unusual instrument behavior, a processing problem, or something we do not yet understand.

Before removing a value, ask whether it is physically possible, whether nearby observations show a similar pattern, whether something was happening at that time, and whether there is enough evidence to make a decision.

Unusual does not automatically mean incorrect.

Step 14: Describe the Data

Now calculate several simple summaries:

```
mean(winter_smoke$concentration)
median(winter_smoke$concentration)
summary(winter_smoke$concentration)
```

mean() calculates the arithmetic mean.

median() calculates the median.

summary() provides several descriptive statistics at once.

Compare the mean and median.

Are they similar?

A numerical summary can tell us about the center, range, and distribution, but it cannot tell us **when** concentrations changed.

Step 15: Visualize Concentration Through Time

Because the measurements were repeatedly collected over several days, a time series graph is a natural first visualization.

We will place:

time on the x axis

and

particle concentration on the y axis

Add another section to your R script:

```
# Create Visualization
```

Then add:

```
plot(
  winter_smoke$date_time,
  winter_smoke$concentration,
  type = "l",
  xlab = "Date / Time",
  ylab = "Particle Concentration (mg/m³)"
)
```

Run the command.

The graph should appear in the **Plots** pane.

Reading the `plot()` Function

Everything inside the parentheses provides information to `plot()`.

These pieces of information are called **arguments**.

```
winter_smoke$date_time  
winter_smoke$concentration  
type = "l"  
xlab = "Date / Time"  
ylab = "Particle Concentration (mg/m3)"
```

Commas separate arguments.

The first argument supplies the x axis data.

The second argument supplies the y axis data.

`type = "l"` tells R to draw a line.

`xlab` supplies the x axis label.

`ylab` supplies the y axis label.

Inside a function, `=` is commonly used to supply a value to a named argument.

Step 16: Look Before Explaining

Study the graph before trying to explain it.

Ask:

- Is concentration constant?
- Are there peaks?
- When do the largest peaks occur?
- Are there periods of relatively low concentration?
- Are some periods more variable?
- Do any patterns appear to repeat?

The first step is **observation**.

Step 17: Interpret What the Data Show

A statement such as:

Particle concentration varied throughout the monitoring period, with periods of relatively low concentration interrupted by higher measured concentrations.

is an observation supported by the dataset.

A statement such as:

Residential wood burning caused the peaks.

is a source attribution claim.

The dataset alone does not prove that.

What Can We Examine?

From the measurements we can examine:

- when concentrations increased
- how high measured concentrations became
- how concentrations changed through time
- whether elevated periods persisted
- how variable the measurements were

What Can We Not Conclude?

From this dataset alone, we cannot determine:

- the exact source of every particle
- why every concentration increase occurred
- whether one source caused a particular peak

Additional information could include wind speed, wind direction, temperature, atmospheric inversions, nearby emission sources, and local heating activity.

Step 18: Document Your Interpretation in the R Script

Your R script can contain more than code.

We can also use **comments** to document what we observed and how we interpreted the results.

Return to your `virtual-lab-01.R` script and add:

```
# Interpretation

# What patterns do you observe in the graph?
# Write your response below.

# What does the evidence show?
# Write your response below.

# What can you NOT conclude from these data?
# Write your response below.
```

Replace the blank comment lines with your own observations and interpretation.

Remember that every line of written text in an R script must begin with #.

For example:

```
# Interpretation

# Particle concentrations changed throughout the monitoring period.
# Several periods had higher measured concentrations than the surrounding
# observations.
# The measurements show when concentrations changed, but these data alone
# cannot identify the source responsible for each increase.
```

Because each line begins with #, R recognizes the writing as a **comment** and does not try to execute it as code.

! Why Not Use Triple Backticks?

Triple backticks are used in **Markdown and Quarto documents**, not for writing paragraphs inside a standard .R script.

Later in SHM 377, you can choose to begin creating **Quarto reports**, you will be able to write paragraphs normally and place R code inside separate code blocks.

For now:

R script: use # for written notes and interpretation.

Quarto report: write normal paragraphs and place R code in code blocks and render .html, .pdf files.

Step 19: Export and Save Your Work

Export the Plot

From the **Plots** pane:

1. Select **Export**.
2. Choose **Save as Image** or **Save as PDF**.
3. Navigate to your `virtual-lab-01` folder.
4. Give the figure a descriptive name.
5. Save it.

For example:

```
winter-smoke-concentration-time-series.jpg
```

Your lab folder may now look like:

```
SHM-377/  
  virtual-lab-01/  
    winter-smoke-2019-12-27_to_2019-12-31.csv  
    virtual-lab-01.R  
    winter-smoke-concentration-time-series.jpg
```

Review Your R Script

A simple version of the important reproducible steps should look like:

```
# Virtual Lab 01  
# Your Name  
# Date  
  
# Import Data  
  
winter_smoke <- read.csv(  
  "winter-smoke-2019-12-27_to_2019-12-31.csv"  
)  
  
# Convert Date and Time  
  
winter_smoke$date_time <- as.POSIXct(  
  winter_smoke$date_time  
)  
  
# Create Visualization  
  
plot(  
  winter_smoke$date_time,  
  winter_smoke$concentration,  
  type = "l",  
  xlab = "Date / Time",  
  ylab = "Particle Concentration (mg/m³)"
```

```
)

# Interpretation

# What patterns do you observe in the graph?
# Write your response below.

# What does the evidence show?
# Write your response below.

# What can you NOT conclude from these data?
# Write your response below.
```

During the analysis, you also used exploratory commands such as:

```
getwd()
list.files()
head(winter_smoke)
tail(winter_smoke)
names(winter_smoke)
dim(winter_smoke)
str(winter_smoke)
min(winter_smoke$concentration)
max(winter_smoke$concentration)
mean(winter_smoke$concentration)
median(winter_smoke$concentration)
summary(winter_smoke$concentration)
```

Those commands helped you evaluate and understand the dataset.

Use the Console to investigate. Use the script to preserve the analysis you want to reproduce.

Why Units Matter

A number without a unit can be difficult or impossible to interpret correctly.

For this dataset, particle concentration is reported in:

milligrams per cubic meter (mg/m³)

The CSV variable is simply called `concentration`, so the variable name itself does not tell us the unit.

Measurement information may come from instrument settings, field notes, data dictionaries, project documentation, sampling protocols, laboratory reports, instrument manuals, or knowledge of the monitoring method.

Always ask:

What was measured, how was it measured, and what units were used?

! Never assume the units

Do not infer a unit only from numerical values. Confirm the unit from reliable measurement or project information.

The SHM 377 Data Workflow

Throughout this course, we will repeatedly use a workflow like this:

```
Question
↓
Data
↓
Check
↓
Describe / Analyze
↓
Visualize
↓
Interpret
↓
Communicate / Manage
```

The specific commands will change.

The reasoning process will remain similar.

Connecting Data to Hazardous Materials Management

```
Material
↓
Properties
↓
Source / Activity
↓
Release
↓
Environmental Medium
↓
Transport
```

↓
Exposure
↓
Dose
↓
Effects

Monitoring provides evidence about parts of that system.

Data analysis helps us move from measurements toward evaluation and decision making.

Hazardous Materials Problem

↓
Measurement
↓
Data
↓
Evaluation
↓
Interpretation
↓
Management / Control

You Do Not Need to Memorize Everything

Today you encountered functions such as:

```
getwd()  
list.files()  
read.csv()  
head()  
tail()  
names()  
dim()  
str()  
as.POSIXct()  
min()  
max()  
mean()  
median()  
summary()  
plot()
```

You do not need to memorize all of them immediately.

What matters more is understanding **why** you are using them.

Ask yourself:

What is my question?

Where are my files?

Can R find them?

Did the data import correctly?

What does my dataset contain?

Do the measurements look reasonable?

How can I describe them?

How can I visualize them?

What does the evidence show?

What can I not conclude?

What You Accomplished

In this first virtual lab, you:

- installed and distinguished R and RStudio
- downloaded a CSV dataset from GitHub
- organized course and lab files on your computer
- created and saved an R script
- learned the major RStudio panes
- distinguished the Console from an R script
- used comments to organize your script
- checked the working directory
- imported environmental monitoring data
- inspected and prepared the dataset
- calculated basic descriptive statistics
- created a time series visualization
- interpreted the results
- distinguished measurement from source attribution

The Main Idea

We are not learning R for the sake of learning R.

We are learning how to use data to investigate, evaluate, and communicate hazardous materials problems.

Where We Go Next

In later virtual labs, you will use the same basic workflow with new hazardous materials questions and datasets.

You already know the foundation:

```
Organize files
↓
Open RStudio
↓
Create an R script
↓
Import data
↓
Inspect
↓
Analyze
↓
Visualize
↓
Interpret
↓
Communicate
```

We do not need to learn everything at once.

We start with the question, organize the information we need, and use the data to work toward an answer.

© 2026 Yoni Rodriguez. All rights reserved.